第四章　接口与继承

章节概述

一、继承
- 1.父类和子类的联系与区别
- 2.继承的具体例子
- 3.继承的优点和缺点

二、抽象类
- 1.抽象方法和抽象类的定义
- 2.抽象类的具体例子
- 3.抽象类的作用
- 4.一些需要注意的点

三、接口
- 1.接口的定义
- 2.接口的具体例子

四、抽象类与接口的对比

五、多态与重写
- 1.多态的定义、好处、使用多态的必要条件
- 2.实现多态的 3 种方式

六、Java 的多继承问题
- 1.Java 不支持多继承的原因
- 2.Java 如何实现多继承效果

七、超类与 super 关键字
- 1.Object 超类
- 2.equals 方法
- 3.super 关键字

八、异常的继承框架
- 1.异常的定义
- 2.异常的继承框架
- 3.自定义异常
- 4.异常的具体例子

一、继承

1.父类和子类的联系与区别

- 父类：刻画子类共有的属性和方法
- 子类：复用父类的属性和方法，可以增加一些新的属性和方法
- 子类通过extends关键字继承父类

2.继承的具体例子（关键：extends和super）

```
1. public class Person{
2.     private String name;
3.     private int age;
4.     public Person(String name, int age){
5.         this.name = name;
6.         this.age = age;
7.     }
8.     public String getName(){
9.         return name;
10.    }
11.    private String setName (String name){
12.        this.name = name;
13.    }
14.}
```

```
1. public class Student extends Person{
2.     super(name, age);
3.     String school;
4.
5.     public string getSchool(){
6.         return this.school;
7.     }
8. }
```

一、继承

3.继承的优点和缺点

- 优点：提高了代码的可复用性，构成了多态的基础
- 缺点：让类的耦合度增强

二、抽象类

1.抽象方法和抽象类的定义

- 抽象方法：一个类中若有尚不确定的方法，将其定义为抽象方法
- 抽象类：拥有抽象方法的类

2.抽象类的具体例子 （关键：abstract）

```
1. //定义一个抽象类
2. public abstract class Person{
3.     //普通方法
4.     public String getName()
5.     {
6.         return name;
7.     }
8.     //抽象方法
9.     //没有方法体，用abstract做修饰
10.     public abstract void getMission();
11.}
12.
13.Person P = new Person();
14.// 编译错误，抽象类不能实例化
```

3.抽象类的作用

- 抽象方法定义了规范
- 只能被继承，保证子类实现其定义的抽象方法
- 可用于实现多态

4.一些需要注意的点

- 抽象类里的抽象方法数≥0
- 含有抽象方法的类必须是抽象类
- 抽象类不能实例化
- 继承抽象类后未实现其定义的抽象方法，则此类仍为抽象类

三、接口

1.接口的定义

- 接口是一系列方法的声明，是一些抽象的集合。接口没有字段，所有方法都是抽象方法。

2.接口的具体例子（关键：interface和implements）

```
1. //定义一个抽象类
2. public abstract class Person{
3.     //抽象方法
4.     public abstract void getDuty();
5.     public abstract void getMission();
6. }
```

```
1. //定义一个接口
2. interface Person{
3.     //抽象方法
4.     void getDuty();
5.     void getMission();
6. }
```

- 当一个具体的class去实现一个interface时，使用implements关键字

```
public class Student implements Person{
    private String name;
    private int age;
    @Override
    public void getDuty(){
        System.out.println("好好学习");
    }
    @Override
    public void getMission(){
        System.out.println("为中华之崛起而读书");
    }
}
```

四、抽象类与接口的对比

比较重点，要记住表格内容

对比	抽象类	接口
抽象方法	抽象方法数≥0	接口内的方法必须全部为抽象方法
具体方法	可以有具体方法	不允许有具体方法
字段	可以定义字段	无字段
继承	只能extends一个class	可以implements多个interface
构造方法	有构造方法	无构造方法
是否可以实例化	不能实例化	不能实例化

根据以上几个方面的对比，接口不是特殊的抽象类，二者有很大的区别。

五、多态与重写

1.多态的定义、好处、使用多态的必要条件

- 定义：多态是同一个行为具有多种表现形态的能力
- 好处：减少耦合，增强可替换性，增强可扩展性，提高灵活性
- 使用多态的必要条件：继承，重写，父类引用指向子类

2.实现多态的3种方式

（1）重写

```
1. public class Person{
2.     private String name;
3.     private int age;

4.     public void getTarget(){
5.         System.out.println( “美好生活” );
6.     }
7. }
```

```
1. public class Student extends Person{
2.     @Override
       public void getTarget(){
3.         System.out.println(“功夫到家");
4.     }
5. }
```

```
1. public class Teacher extends Person{
2.     @Override
       public void getTarget(){
3.         System.out.println(“传道授业解惑");
4.     }
5. }
```

```
public static void show_target(Person P_x) {
     System.out.print(" 目标: ");
     P_x.getTarget();
 }
Person P1= new Student();
Person P2= new Teacher();
show_target(P1);
show_target(P2);
```

（2）抽象类和抽象方法

```
1. //定义一个抽象类
2. public abstract class Person{
3.     ……
4.     //抽象方法
5.     //没有方法体，用abstract做修饰
6.     public abstract void getTarget();
7.     ……
8. }
```

```
public class Scientist extends Person{
    @Override
    public void getTarget(){
        System.out.println("勇攀科学高峰");
    }
}
```

```
public static void main(String[] args) {
    Person Wang = new Engineer();
    Wang.getTarget();
    Person Li = new Scientist();
    Li.getTarget();
}
```

（3）接口

```
//接口：目标
public interface GetTarget(){
    void getTarget();
}
```

```
//Talent类继承了Person类，且实现了目标接口
public class Talent extends Person implements GetTarget(){
    @Override
    public void getTarget(){
        System.out.println("打造国之重器");
    }
}
```

```
public static void main(String[] args) {
    Person Zhang= new Talent();
    Zhang.getTarget();
    Person Ma = new Entrepreneur();
    Han.getTarget();
}
```

- 继承/实现：extends/implements
- 重写：@Override
- 实例化（父类引用指向子类）：父类名 引用名 = new 子类名();

六、Java的多继承问题

这部分只需要了解即可，有个印象就好

1.Java不支持多继承的原因

若子类继承的多个父类当中，有相同的属性或相同的方法，则在使用该属性或方法时不能判断它来自哪个父类，造成混淆。

2.Java如何实现多继承效果

- 内部类：在一个类的内部定义一些类，这些内部的类分别去继承不同的类
- 接口：一个接口extends多个接口，再用类去实现该接口

七、超类与super关键字

1.Object超类

- 是所有类的始祖
- 可以被显式或隐式继承
- Object类型的变量可以引用任何对象，但对其中内容具体操作时需要强制类型转换

2.equals方法

- 比较基本数据类型时，比较的是取值是否相等
- 比较引用数据类型时，比较的是地址是否相等（即是否指向同一对象）

3.super关键字

super关键字的功能

- 在构造方法中显式调用父类的构造方法
- 访问父类的成员方法和变量

```java
public class Person {
    public Person(String name, int age) {
    }
}
public class Student extends Person {
    public Student(String name, int age, String birth) {
        super(name, age); // 调用父类中含有2个参数的构造方法
    }
}
```

- 使用 super 访问父类中的成员与 this 关键字的使用相似，只不过它引用的是子类的父类，语法格式如：`super.X`
  - 其中，X是父类中的属性或方法。
- 当父类和子类都具有相同的方法名时，可以使用 super 关键字访问父类的方法。

```java
class Person {
    void getTarget() {
        System.out.println("美好生活");
    }
}
class Student extends Person {
    void getTarget() {
        System.out.println("功夫到家");
    }
    void display() {
        getTarget();
        super.getTarget();
    }
}
```

调用当前student类的getTarget() 方法

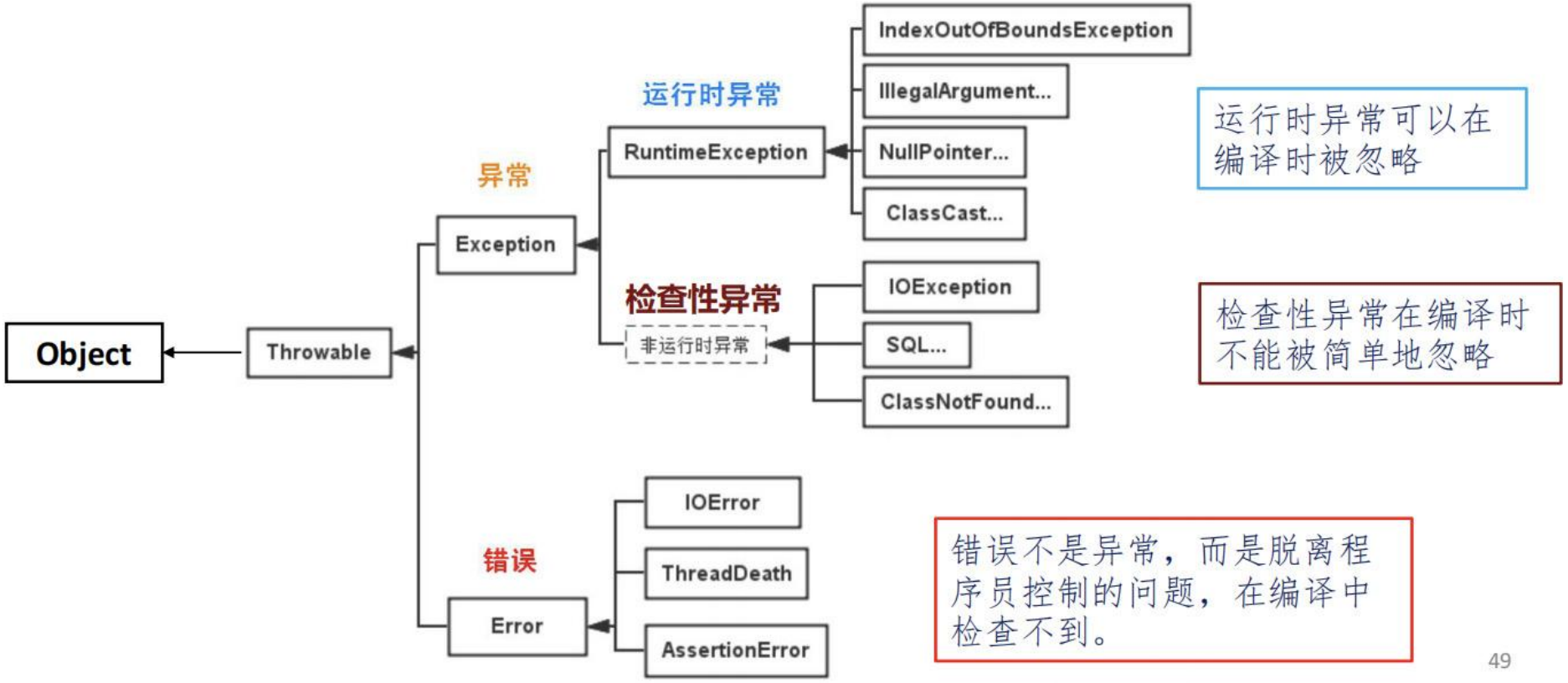
调用父类Person类的getTarget() 方法

八、异常的继承框架

1.异常的定义

- 异常是程序运行时发生的不被期望的事件

2.异常的继承框架：Throwable是异常体系的根



3.自定义异常

- 检查性异常：继承Exception类
- 运行时异常：继承RuntimeException类

4.异常的具体例子

```
1. public class MyException extends Exception {
2.     public MyException() {
3.         super();
4.     }
5.     public MyException(String str) {
6.         super(str);
7.     }
8. }
```

```
try {
    score = input.nextInt();    // 获取成绩
    if(score < 0) {
        throw new MyException("成绩为负！");
    } else if(score > 100) {
        throw new MyException("你的优秀已经溢出了！");
    } else {
        System.out.println("你的成绩为："+score);
    }
} catch(InputMismatchException e1) {
    System.out.println("输入的成绩不是数字！");
} catch(MyException e2) {
    System.out.println(e2.getMessage());
}
```

小练习

1.判断题：接口是特殊的抽象类。

2.判断题：在JAVA中，可以用内部类方式或者接口方式来实现多继承。

3. 在右图的例子中，输出的结果“学生年龄：”是？

A 18

B 39

C 不清楚

```
1. class Person {
2.     int age = 39;
3. }
4. class Student extends Person {
5.     int age = 18;
6.     void display() {
7.         System.out.println("学生年龄:
   " + super.age);
8.     }
9. }
10.class Test {
11.    public static void main(String[] args) {
12.        Student stu = new Student();
13.        stu.display();
14.    }
15.}
```

小练习答案

1.错　2.对　3.B